

1. INTRODUCTION

Commercial contracts are the backbone of nearly every business transaction, yet reviewing them remains a slow, expensive, and highly manual process. Studies of legal workflows consistently report that law firms and in-house counsel spend a very large share of their time reading and re-reading contracts to locate a handful of clauses — limitation of liability, termination rights, exclusivity, governing law — that materially affect risk. As transaction volumes grow, purely manual review does not scale, and important clauses are sometimes missed simply because a reviewer runs out of time or attention.

Natural Language Processing (NLP) and Machine Learning (ML) offer a practical way to reduce this burden by automatically identifying and categorising the clauses that matter. ClauseIQ is a mini project that applies classical, well-understood ML techniques — rather than large deep-learning or Retrieval-Augmented Generation (RAG) systems — to the problem of contract clause classification and intelligent clause search. The project deliberately favours interpretable, computationally light models (TF-IDF vectorisation combined with Naive Bayes, Support Vector Machines, Logistic Regression and Random Forest) over transformer-based architectures, in keeping with the scope expected of a semester mini project. A more ambitious deep-learning / RAG-based system (originally proposed as “DocuMind AI”) was considered but was found to be too broad for this stage and has been deferred to a possible fourth-semester main project.

The system is built around the Contract Understanding Atticus Dataset (CUAD), a corpus of 510 commercial contracts drawn from the U.S. Securities and Exchange Commission’s EDGAR database and expert-annotated by lawyers and law students associated with The Atticus Project. CUAD identifies 41 categories of clauses that are routinely reviewed in corporate transactions — such as Governing Law, Termination for Convenience, Non-Compete, and Cap on Liability — spanning more than 13,000 individually annotated clause instances. This makes it one of the largest, highest-quality, expert-labelled legal NLP resources publicly available, and a natural fit for a supervised clause-classification task.

ClauseIQ has two complementary goals. The first is classification: given the text of a clause (or a labelled span extracted from a contract), predict which of the 41 CUAD categories it belongs to, so that a reviewer can be automatically shown “this looks like a Non-Compete clause” instead of reading the whole document. The second is intelligent search: given a natural-language query (e.g. “find clauses that cap our liability”), retrieve the most relevant clauses across the corpus using TF-IDF vector representations and cosine similarity, without requiring an exact keyword match. Together, classification and search form a lightweight “contract intelligence” pipeline that mirrors, at a much smaller scale, the kind of tooling used in commercial contract-review products.

2. SUPPORTING LITERATURE

2.1. Literature Review

Paper 1 — Quevedo E., Cerny T., Rodriguez A., Rivas P., Yero J., Sooksatra K., Zhakubayev A., Taibi D. (2024)

“Legal Natural Language Processing From 2015 to 2022: A Comprehensive Systematic Mapping Study of Advances and Applications.” IEEE Access, vol. 12, pp. 145286–145317. DOI: 10.1109/ACCESS.2023.3333946.

The surge in legal-text production has increased the workload of legal professionals, and the specialised, dense language of legal documents makes many legal-NLP tasks repetitive and time-consuming to perform manually. Although Legal NLP has advanced steadily since 2015 with the growth of Deep Learning and, more recently, Large Language Models, no systematic account of this progress existed up to 2022. This paper addresses that gap through a Systematic Mapping Study (SMS): a structured literature search across four reputable academic indexers that began with 536 candidate papers and, after applying explicit inclusion/exclusion criteria, filtered them down to 75 pivotal articles. The authors then categorised and sub-categorised these studies by research problem, revealing that Legal NLP work concentrates heavily on a handful of task families — most notably Multiclass Classification, Summarization, and Question Answering — and that both classical, feature-based ML pipelines (e.g., TF-IDF with Naive Bayes or SVM) and deep, transformer-based methods are represented across these tasks. The study also highlights recurring resource gaps: a shortage of curated, well-labelled legal datasets, limited legal ontologies, and persistent data-accessibility problems, all of which constrain reproducible progress in the field.

For ClauseIQ, this systematic mapping study is valuable in three ways. First, it independently confirms that Multiclass Classification of legal text — precisely the task ClauseIQ performs across CUAD’s 41 clause categories — is one of the most actively researched problem families in Legal NLP, not a niche or unusual framing of the problem. Second, because the SMS spans both classical ML and deep-learning solutions to these tasks without concluding that one dominates, it supports treating a TF-IDF plus classical-ML pipeline as a legitimate, literature-grounded starting point rather than an outdated shortcut. Third, the paper’s explicit identification of dataset scarcity and curation gaps as an open challenge underlines why CUAD — a large, expert-annotated, well-structured resource of over 13,000 clause instances — is a particularly appropriate dataset choice for a project at this scope, and why category-level data scarcity (i.e., class imbalance) should be treated as an expected, literature-documented characteristic of legal multiclass classification rather than a defect specific to CUAD.

TITLE	Legal Natural Language Processing From 2015 to 2022: A Comprehensive Systematic Mapping Study of Advances and Applications
AREA OF WORK	Systematic Mapping Study (SMS) of Legal NLP research published between 2015 and 2022, covering task types, methods, datasets and open challenges.
PROBLEM STATEMENT	No systematic account existed of how Legal NLP progressed between 2015 and 2022, despite rapid growth driven by Deep Learning and LLMs, leaving researchers without a structured view of methods, gaps and opportunities in the field.
DATASET	Not a single experimental dataset; a literature corpus of 536 candidate papers, sourced from four academic indexers and filtered to 75 pivotal primary studies using explicit inclusion/exclusion criteria.
METHODOLOGY	Systematic Mapping Study (SMS) methodology: structured multi-indexer search, screening against inclusion/exclusion criteria, and categorisation/sub-categorisation of the retained studies by research problem.
ALGORITHMS / TECHNIQUES	Comparative synthesis of methods used across the reviewed studies, spanning classical feature-based ML (TF-IDF with Naive Bayes, SVM, Logistic Regression), deep-learning architectures, and, in later studies, Large Language Models.
RESULTS	Legal NLP research concentrates on a small number of dominant task families — Multiclass Classification, Summarization and Question Answering — addressed by a diverse mix of classical and deep methods; 75 pivotal articles were catalogued and mapped by task and technique.
ADVANTAGES	Provides a structured, evidence-based taxonomy of Legal NLP tasks and methods; explicitly documents open challenges (dataset curation, ontology gaps, data accessibility) that later work, including ClauseIQ, can address.
LIMITATIONS	As a mapping study it reports no new empirical benchmark results of its own; coverage is bounded to 2015–2022 and therefore excludes the most recent LLM-era work; findings are dependent on the coverage of the four chosen indexers and the authors’ inclusion criteria.
FUTURE SCOPE	Calls for more curated legal datasets and ontologies, improved data accessibility, and continued systematic tracking of Legal NLP progress as LLM-based methods mature.

RELEVANCE TO CLAUSEIQ	Confirms multiclass legal-text classification as a core, well-studied Legal NLP problem; legitimises a classical TF-IDF + ML pipeline as a literature-grounded design choice; frames CUAD’s scale and annotation quality, and the resulting class imbalance across 41 categories, as consistent with documented characteristics of the field rather than a project-specific weakness.
------------------------------	---

2.1

Table 2.2: Summary of Paper 1

Paper 2 — Siino M., Falco M., Croce D., Rosso P. (2025)

“Exploring LLMs Applications in Law: A Literature Review on Current Legal NLP Approaches.” IEEE Access, vol. 13, pp. 18253–18276. DOI: 10.1109/ACCESS.2025.3533217.

This is a systematic literature review (61 selected publications, 2017–2023) that maps how Transformer/LLM-based NLP has been applied across the legal domain, including legal document analysis, case prediction, legal search, regulatory compliance and — most relevant to ClauseIQ — contract review. The survey traces a clear progression from classical, feature-based text classification (bag-of-words / TF-IDF with SVM or Naïve Bayes) toward BERT-family and, more recently, LLM-based approaches, and it explicitly discusses CUAD as one of the standard benchmark datasets used for contract clause identification. The review follows a PRISMA-style protocol with clearly defined research questions and a CASP-based quality-assessment checklist, and it reports a clear upward trend in both publication volume and task diversity across the surveyed period, alongside a persistent trade-off between the interpretability and low cost of classical ML pipelines and the higher (but more expensive and less transparent) accuracy typically reported for transformer-based models.

For ClauseIQ, the survey is useful in two ways: it confirms that clause-level classification and semantic clause search are recognised, well-studied sub-tasks within Legal NLP, with CUAD named explicitly as a benchmark dataset for exactly this purpose, and it documents the accuracy/interpretability/cost trade-off that directly supports the project’s decision to begin with a classical ML baseline before considering deep-learning or LLM-based extensions.

TITLE	Exploring LLMs Applications in Law: A Literature Review on Current Legal NLP Approaches
AREA OF WORK	Systematic literature review of NLP/LLM applications across the legal domain, including contract review, legal document classification, case prediction and legal search.
PROBLEM STATEMENT	The rapid rise of LLM-based legal NLP tools has outpaced structured understanding of where these approaches are actually applied, how they compare to earlier classical methods, and where meaningful gaps and risks remain.

DATASET	61 publications (2017–2023) selected via database search and backward snowballing; referenced benchmark datasets include CUAD, ContractNLI and other legal NLP corpora.
METHODOLOGY	PRISMA-style systematic review with defined research questions, explicit inclusion/exclusion criteria and a CASP-based quality-assessment checklist applied to each selected study.
ALGORITHMS / TECHNIQUES	Comparative discussion of TF-IDF / classical ML classifiers, BERT-family encoders, and generative LLMs as applied across the reviewed studies.
RESULTS <i>metrics/ score</i>	Identifies a clear upward trend in publication volume and task diversity, and documents accuracy/interpretability trade-offs between classical and transformer-based legal NLP methods; explicitly names CUAD as a standard contract-clause-identification benchmark.
ADVANTAGES	Provides a structured taxonomy of legal NLP tasks and confirms contract clause classification / search as an established, well-motivated research problem, directly naming the dataset ClauseIQ uses.
LIMITATIONS	As a review, it synthesises reported results rather than independently reproducing them; the 2017–2023 window may not capture the very latest LLM releases; conclusions on the classical-vs-LLM trade-off are qualitative rather than a single controlled benchmark.
FUTURE SCOPE	Calls for more standardised evaluation protocols and greater attention to explainability and cost-effective (non-LLM) approaches for practical deployment.
RELEVANCE TO CLAUSEIQ	Directly names CUAD as a contract-clause benchmark dataset, validating ClauseIQ’s dataset choice; documents the classical-ML-vs-LLM interpretability/cost trade-off that justifies starting with TF-IDF and classical classifiers rather than a transformer/RAG pipeline.

2.2
Table 2.4. Summary of Paper 2

Paper 3 — Kutbi M. (2023)

“Named Entity Recognition Utilized to Enhance Text Classification While Preserving Privacy.” IEEE Access, vol. 11. DOI: 10.1109/ACCESS.2023.3325895.

Text preprocessing is a critical step in any NLP pipeline, and named entities — person names, organisations, addresses, dates — pose a particular problem: removing them discards potentially useful structure, while leaving them as raw tokens adds noise, inflates the feature space, and can leak identifying information. This paper proposes a simple, model-agnostic alternative: using a Named Entity Recognition (NER) tagger (spaCy) to detect named entities and replace each one with a generic placeholder for its entity type (e.g., PERSON, ORG, GPE) rather than deleting it or

leaving it verbatim. The approach is evaluated on three text-classification datasets, including one directly relevant to ClauseIQ: an in-house collected legal dataset of Limited Partnership Agreement (LPA) contract paragraphs spanning 20 classes, alongside two public non-legal datasets (including movie reviews) used to test whether the technique generalises beyond the legal domain. Across these datasets, replacing entities with their type label measurably improved downstream text-classifier accuracy and reduced feature-vector dimensionality compared with leaving raw entity text in place, while simultaneously anonymising sensitive identifying details.

This paper is the most directly applicable of the three to ClauseIQ’s own preprocessing design, since it operates on legal contract paragraphs (an LPA corpus) using the same broad workflow ClauseIQ follows: clean text, vectorise it, and classify it with a standard classifier. CUAD clauses, like the LPA paragraphs studied here, are dense with party names, company names, dates and addresses that carry little category-discriminating signal for a 41-way classifier but do carry privacy and dimensionality costs if left untreated — exactly the problem this paper’s NER-based preprocessing step is designed to solve, and a concrete, literature-grounded extension that ClauseIQ’s cleaning pipeline can adopt in its implementation sprints.

TITLE	Named Entity Recognition Utilized to Enhance Text Classification While Preserving Privacy
AREA OF WORK	NLP text-preprocessing technique that uses Named Entity Recognition to improve text-classifier accuracy while preserving the privacy of sensitive identifying information.
PROBLEM STATEMENT	Standard text preprocessing either removes named entities (losing potentially useful structure) or leaves them as raw tokens (adding noise, inflating dimensionality, and risking privacy leakage of names, organisations and other identifying details).
DATASET	Three text-classification datasets: an in-house Limited Partnership Agreement (LPA) legal-contract corpus (20 classes, 319 paragraphs) and two public non-legal datasets, including a movie-review dataset, used to test cross-domain generality.
METHODOLOGY	Preprocessing pipeline using spaCy’s NER model to detect named entities and replace each with a generic entity-type placeholder (e.g., PERSON, ORG, GPE, LOC) instead of deleting or retaining the raw text; classifiers trained and compared with and without this preprocessing step.
ALGORITHMS / TECHNIQUES	spaCy Named Entity Recognition; standard feature-based text classification pipeline; comparison of vector-space dimensionality and classifier accuracy with vs. without entity-type replacement.

RESULTS metrics/ score	Entity-type replacement improved downstream classification accuracy and reduced feature-vector dimensionality relative to leaving raw entities in place, across both the legal (LPA) and non-legal datasets tested.
ADVANTAGES	Simple, model-agnostic preprocessing step that simultaneously improves classifier accuracy, reduces dimensionality, and anonymises sensitive identifying information — directly applicable where contracts name real parties and organisations.
LIMITATIONS	Entity-type replacement can occasionally discard genuinely useful signal (e.g., a specific name that meaningfully affects classification, as observed in the movie-review experiments); results depend on the coverage and accuracy of the underlying NER tagger, which may not perfectly fit specialised legal terminology.
FUTURE SCOPE	Extending entity-type replacement to domain-tuned NER models and evaluating the approach across a wider range of legal-document types.
RELEVANCE TO CLAUSEIQ	Operates on legal contract paragraphs using the same TF-IDF-style classical pipeline ClauseIQ follows; directly motivates treating party names, organisation names and dates in CUAD clauses via NER-based generalisation rather than leaving them as noisy, privacy-sensitive tokens, as a concrete refinement for ClauseIQ’s data-cleaning stage.

2.3

Table 2.6. Summary of Paper 3

2.2. Literature Review Summary

Title	Area of Work	Dataset	Methodology	Algorithms	Advantages	Limitations	Relevance to ClauseIQ
Legal NLP 2015–2022: Comprehensive SMS (IEEE Access, 2024)	Systematic mapping of Legal NLP tasks/-gaps, 2015–2022	536 papers → 75 pivotal articles, 4 indexers	SMS: search, screening, categorisation by problem	Classical ML, Deep Learning, LLMs (surveyed)	Evidence-based taxonomy; flags dataset/ontology gaps	No new benchmarks; 2015–2022 only; indexer-dependent	Confirms multiclass legal classification as core task; backs classical TF-IDF baseline
Exploring LLMs Applications in Law: Review (IEEE Access, 2025)	LLM/NLP in law, incl. contract review, legal search	61 papers (2017–2023); benchmarks incl. CUAD	PRISMA-style review, CASP quality assessment	TF-IDF/classical ML, BERT-family, LLMs (surveyed)	Names CUAD as clause benchmark; documents cost/accuracy trade-off	Qualitative synthesis; window ends 2023	Validates CUAD choice; backs classical-ML-first strategy
NER Utilized to Enhance Text Classification While Preserving Privacy (IEEE Access, 2023)	NER preprocessing for private, accurate text classification	Legal LPA contracts (20 classes) + 2 public datasets	Replace entities with type placeholders pre-classification	spaCy NER + feature-based classifiers	Improves accuracy, cuts dimensionality, preserves privacy	May drop useful signal; depends on NER tagger quality	Directly informs cleaning of party names/dates in CUAD clauses

2.4

Table 2.8. Overall Summary of All Papers

Not readable, Reduce the number of columns as most of the informations are available in prior tables

2.3. Findings and Proposals

The three reviewed papers, taken together, provide a coherent, literature-grounded case for every major design decision behind ClauseIQ.

Contract clause classification matters because commercial contracts are dense, high-stakes docu-

ments that are still reviewed largely by hand; both Quevedo et al. (2024) and Siino et al. (2025) independently identify multiclass legal-text classification and contract review as core, heavily researched problem families within Legal NLP, confirming that automating clause identification is a recognised and valuable research direction rather than a narrow or artificial framing of the problem. Legal NLP as a field matters for the same underlying reason these surveys exist in the first place: legal language is specialised, high-volume, and consequential, and Quevedo et al. (2024) show that the research community has responded with a steadily growing, increasingly diverse body of work spanning classification, summarisation and question answering.

TF-IDF is a suitable feature representation because it converts short, boilerplate-heavy clauses into a sparse, interpretable numeric space that highlights category-discriminating vocabulary without requiring the large labelled corpora that transformer embeddings typically need; Siino et al. (2025) situate TF-IDF-based pipelines as an established, literature-documented starting point in legal NLP, used well before — and still alongside — BERT-family and LLM-based methods. Classical Machine Learning (Naive Bayes, SVM, Logistic Regression and Random Forest) is selected over deep learning because Quevedo et al. (2024) show classical, feature-based pipelines remain a substantial and actively used category of solution across the Legal NLP literature they mapped, and because Siino et al. (2025) explicitly document a trade-off in which classical ML offers lower cost, faster training and greater interpretability than LLM-based approaches, at the price of some accuracy — an acceptable and, for a scope-controlled mini project, appropriate trade to make. The CUAD dataset is appropriate precisely because it is the dataset Siino et al. (2025) name as a standard benchmark for contract-clause identification, and because its scale and expert annotation address the dataset-curation gap that Quevedo et al. (2024) identify as a recurring weakness in the field; the resulting class imbalance across CUAD’s 41 categories is consistent with the kind of data-scarcity challenge these surveys describe as typical of legal multiclass classification, rather than a flaw specific to this project. Cosine similarity is selected for the intelligent-search module because it is the standard, scale-invariant similarity measure for sparse TF-IDF vector spaces, allowing free-text queries to be matched against clauses using the same feature representation already built for classification, without introducing a second, incompatible modelling paradigm.

Kutbi (2023) contributes a further, concrete refinement: operating on a legal contract corpus (Limited Partnership Agreements) using the same broad TF-IDF-classification workflow ClauseIQ follows, it shows that replacing named entities (party names, organisations, dates) with generic entity-type placeholders before classification improves accuracy, reduces feature-space dimensionality, and preserves the privacy of identifying details. Since CUAD clauses are similarly dense with party names, company names and dates that carry limited category-discriminating value, this finding directly motivates a future improvement to ClauseIQ’s preprocessing pipeline: integrating NER-based entity-type replacement (e.g., via spaCy) into the data-cleaning stage described in Section 3.2, alongside the existing whitespace normalisation and short-span filtering. Beyond this, future improvements consistent with all three papers include periodically re-evaluating CUAD-trained models as newer legal NLP benchmarks and curated datasets be-

come available (per Quevedo et al., 2024), and revisiting the classical-ML-versus-transformer trade-off documented by Siino et al. (2025) once a working classical baseline exists, as a natural bridge toward the fourth-semester main project should a transformer or RAG-based extension be pursued.

Proposal should be given as the final paragraph

3. SYSTEM ANALYSIS

This section documents the machine-learning system analysis for ClauseIQ: dataset analysis, preprocessing, feature and class analysis, visualisation, algorithm study, project pipeline and feasibility. All figures, statistics and code outputs in this section were produced by actually loading and analysing the real CUAD v1 dataset (obtained from the official Atticus Project GitHub repository, <https://github.com/TheAtticusProject/cuad>) rather than a simulated or placeholder dataset.

3.1. Analysis of the Dataset

3.1.1. About the Dataset

The Contract Understanding Atticus Dataset (CUAD) v1 is sourced from the Atticus Project and distributed at <https://github.com/TheAtticusProject/cuad> (and mirrored on Hugging Face Datasets). It contains 510 real commercial contracts collected from the U.S. Securities and Exchange Commission’s public EDGAR filing system, spanning 25 contract types (Distributor Agreements, Franchise Agreements, Licence Agreements, Strategic Alliance Agreements, etc.). Each contract has been manually annotated by law-student and attorney annotators against 41 clause categories that lawyers routinely review during due diligence — for example Governing Law, Anti-Assignment, Cap On Liability, Non-Compete, and Ip Ownership Assignment. In its raw distributed form, CUAD is released as a SQuAD-style question-answering JSON file (CUADv1.json), where each of the 41 categories is phrased as a “question” (e.g. “Highlight the parts of this contract related to ‘Non-Compete’...”) and the answer is one or more text spans (the actual clause text) located inside the contract.

For ClauseIQ’s classification and search use case, this QA-style format was converted into a flat, clause-level table: each row is one annotated clause span, together with the contract it came from and its category label. This conversion was performed directly on the official CUADv1.json file using a Python script (json + pandas), producing the working dataset `clauseiq_dataset.csv` used for all analysis in this report.

Dataset source: <https://github.com/TheAtticusProject/cuad> (Hendrycks, Burns, Chen & Ball, “CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review”, NeurIPS 2021)

- Raw file used: CUADv1.json (SQuAD-style question-answering format)
- Contracts: 510 real commercial contracts sourced from SEC EDGAR
- Clause categories (classes): 41
- Annotated clause spans after flattening and de-duplication: 13,823
- License: Creative Commons Attribution 4.0 (CC BY 4.0)

3.1.2. Explore the Dataset

The flattened clause-level dataset was loaded with pandas and inspected using the same exploratory workflow used for structured tabular data: shape, data types, first records, missing values and summary statistics.

a) Shape and data types

```
df = pd.read_csv('clauseiq_dataset.csv')
print('SHAPE:', df.shape)
print(df.dtypes)
```

```
SHAPE: (13823, 5)
```

```
contract      object
category      object
clause_text   object
char_len      int64
word_count    int64
dtype: object
```

Figure 3.1: Shape and data types of the flattened clause dataset

b) First five records

	clause_text	char_len	word_count	contract	category
0	LIMEENERGYCO_09_09_1999-EX-10-DISTRIBUTOR AGR... DISTRIBUTOR AGREEMENT	21	2		Document Name
1	LIMEENERGYCO_09_09_1999-EX-10-DISTRIBUTOR AGR... Distributor	11	1		Parties
2	LIMEENERGYCO_09_09_1999-EX-10-DISTRIBUTOR AGR... Electric City Corp.	19	3		Parties
3	LIMEENERGYCO_09_09_1999-EX-10-DISTRIBUTOR AGR... City of Illinois L.L.C.	32	5		Parties Electric
4	LIMEENERGYCO_09_09_1999-EX-10-DISTRIBUTOR AGR... Company	7	1		Parties

Figure 3.2: First five rows of the clause-level dataset

c) Missing values and duplicates

Because every row was constructed only from non-impossible, non-empty answer spans in the source JSON, the flattened dataset contains no missing values and no exact duplicate rows, as confirmed below.

```
MISSING VALUES:
contract      0
category      0
clause_text   0
char_len      0
word_count    0
dtype: int64

DUPLICATE ROWS: 0
```

Figure 3.3: Missing value and duplicate-row check

d) Summary statistics

	char_len	word_count
count	13823.000000	13823.000000
mean	262.018882	40.678290
std	285.154804	43.445576
min	1.000000	1.000000
25%	37.000000	5.000000
50%	196.000000	31.000000
75%	365.000000	57.000000
max	3884.000000	479.000000

Figure 3.4: Summary statistics of clause length (characters and words)

Clause length is right-skewed: the median clause is 31 words long, but a long tail of clauses (up to 479 words) corresponds to lengthy definitional or indemnification-style provisions. This motivates using TF-IDF (which is length-normalised) rather than raw word counts as the feature representation.

e) Clauses per contract and per category

```
Clauses per contract (describe):
count    510.000000
mean     27.103922
std      18.120720
min       2.000000
```

25%	14.000000
50%	22.000000
75%	33.000000
max	97.000000

Figure 3.5: Distribution of number of annotated clauses per contract

On average each contract contributes about 27 labelled clauses, but this varies widely (2 to 97), reflecting genuine differences in contract length and complexity across the 25 contract types in CUAD.

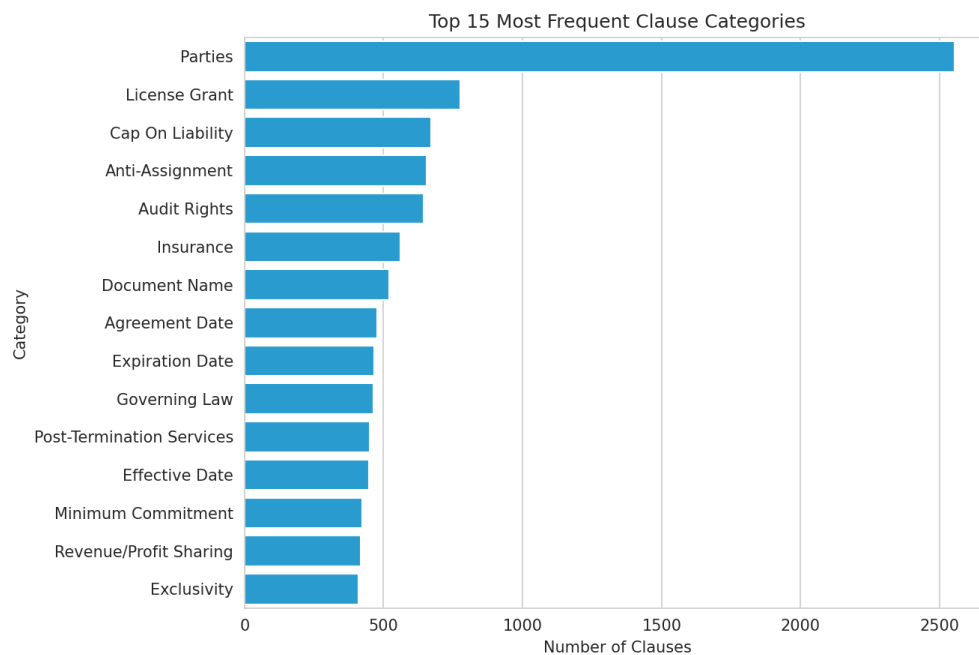


Figure 3.6: Top 15 most frequent clause categories in CUAD

The category distribution is highly uneven: “Parties” alone accounts for 2,554 of the 13,823 clauses (about 18%), followed by “License Grant”, “Cap On Liability”, “Anti-Assignment” and “Audit Rights”, each in the 600–800 range. This class imbalance, discussed further in Section 3.2.3, is a central design consideration for the modelling stage.

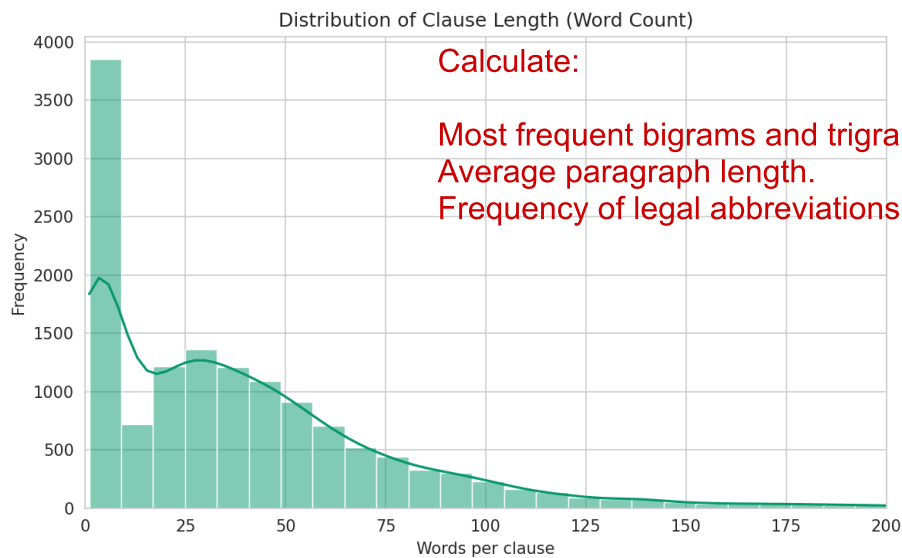


Figure 3.7: Distribution of clause length (word count)

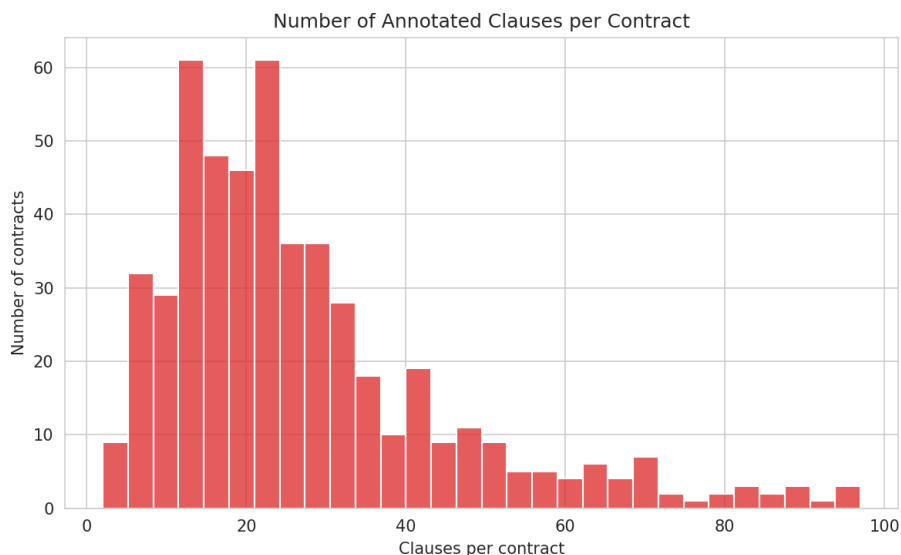


Figure 3.8: Number of annotated clauses per contract

3.2. Data Preprocessing

3.2.1. Data Cleaning

Two cleaning steps were applied to the flattened clause dataset before feature extraction:

- *Text normalisation* — collapsing repeated whitespace/newlines (a common OCR/formatting artefact in the source contracts) and stripping characters outside a legal-text-safe set (letters, digits, and common punctuation such as periods, commas, parentheses, %, \$, and hyphens).
- *Removal of extremely short spans* — clause spans under 3 tokens (e.g. a lone party name or date fragment picked up as a partial answer span) carry little topical signal for a 41-way classifier and were removed.

```
def clean_text(t):
    t = re.sub(r'\s+', ' ', str(t))          # collapse whitespace
    t = re.sub(r"^[^A-Za-z0-9.,;:()$%\\-\\s]", ' ', t) # strip odd artefacts
    return t.strip()

df['clean_text'] = df['clause_text'].apply(clean_text)
df_clean = df[df['clean_text'].str.split().str.len() >= 3]
```

Before cleaning: (13823, 5)
 Removed 1567 clauses with < 3 tokens (11.34%)

 Cleaned dataset shape: (12256, 6)

Figure 3.9: Result of data cleaning and short-span removal

3.2.2. Analysis of Feature Variables

Clause text is converted into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF) with unigrams and bigrams, a vocabulary capped at 5,000 terms, English stop-word removal, and a minimum document frequency of 2 (to discard one-off tokens such as OCR noise or party names).

```
tfidf = TfidfVectorizer(max_features=5000, stop_words='english',
                        ngram_range=(1,2), min_df=2)
X = tfidf.fit_transform(df_clean['clean_text'])
print('TF-IDF matrix shape:', X.shape)
print('Sparsity: {:.4f}%'.format(
    100 * (1 - X.nnz / (X.shape[0]*X.shape[1]))))
```

TF-IDF matrix shape: (12256, 5000)
 Sparsity: 99.5187%

Top 20 TF-IDF terms overall:

agreement	449.05
shall	394.78
party	353.97
term	211.92
company	203.05
date	171.49
section	160.32

written	158.05
right	153.40
license	144.56

Figure 3.10: TF-IDF feature matrix statistics and top terms

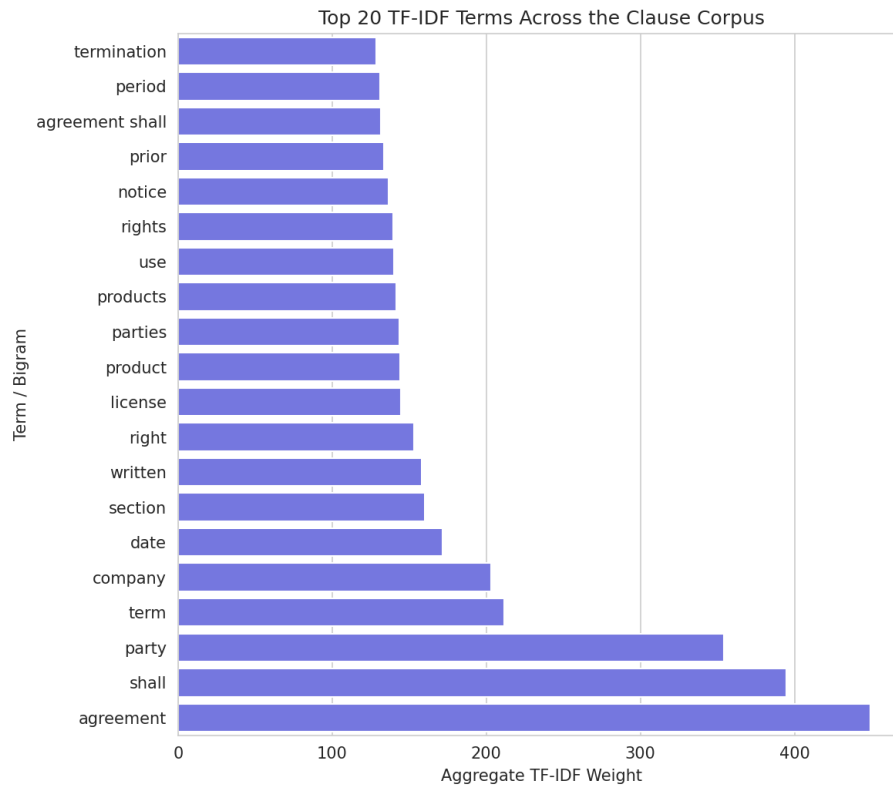


Figure 3.11: Top 20 TF-IDF terms across the clause corpus

As expected for legal text, the highest-weighted terms are boilerplate contract vocabulary (“agreement”, “shall”, “party”, “section”). These generic terms are shared across almost every category, so the classifiers must rely on more category-specific bigrams (e.g. “agreement shall”, “prior written”) and rarer discriminating terms to separate the 41 classes — this is one reason bigrams were included in the TF-IDF configuration rather than unigrams alone.

3.2.3. Analysis of Class Variables

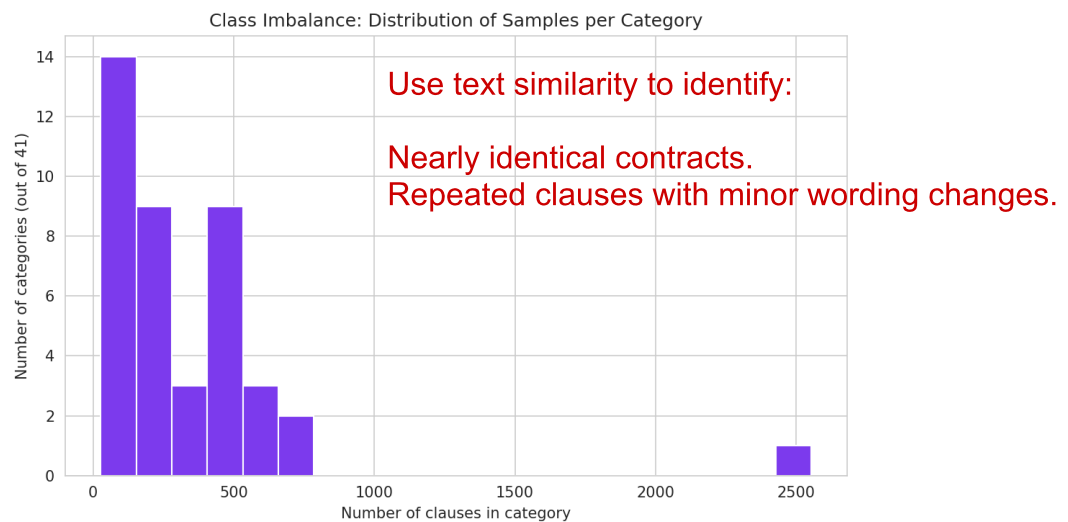


Figure 3.12: Class imbalance: number of categories by sample-count bucket

The 41 clause categories are strongly imbalanced. The largest class (“Parties”, 2,554 samples) has roughly 95 times more examples than the smallest class (“Price Restrictions”, 27 samples). Four categories fall below 40 samples after cleaning:

Categories with fewer than 40 samples:	
Third Party Beneficiary	39
Most Favored Nation	38
Unlimited/All-You-Can-Eat-License	32
Price Restrictions	27

Figure 3.13: Rare categories identified for class-imbalance handling

Following the imbalance-related challenges documented across the Legal NLP literature reviewed by Quevedo et al. (2024) in Section 2, the modelling stage will address this imbalance using class-weighted loss (`class_weight='balanced'` in scikit-learn), stratified train/test splitting so every category is represented in both splits, and macro-averaged F1-score (rather than plain accuracy) as the primary evaluation metric, since accuracy alone would be dominated by the majority “Parties” class.

3.3. Data Visualization

The following visualisations summarise the shape of the CUAD-derived clause dataset used for ClauseIQ.

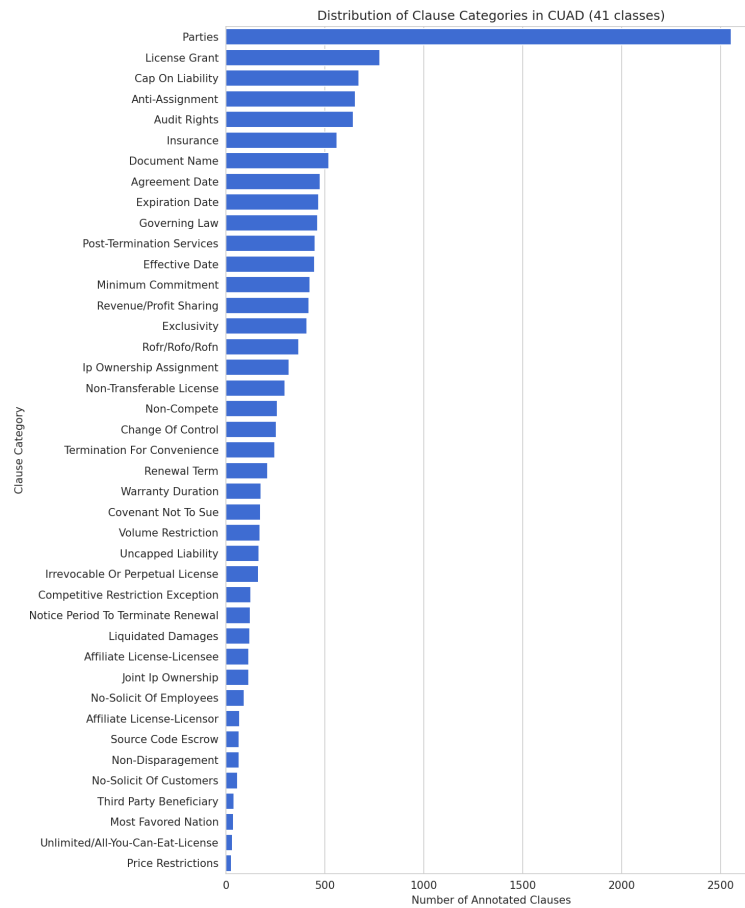


Figure 3.14: Full distribution of all 41 clause categories

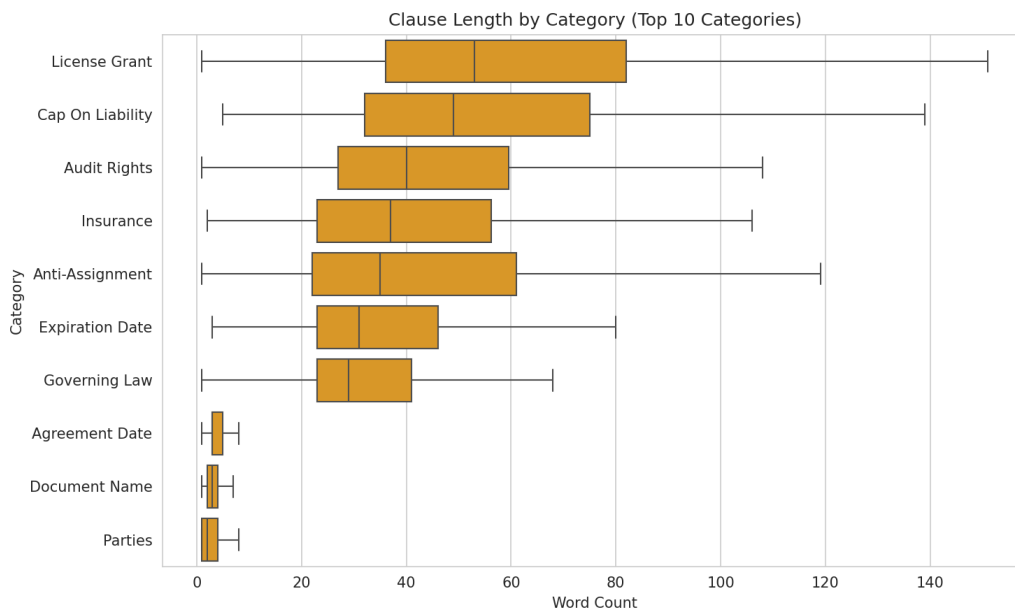


Figure 3.15: Clause length by category (top 10 most frequent categories)

Clause length also varies meaningfully by category — e.g. “Audit Rights” and “Insurance” clauses tend to be longer and more detailed than short factual spans such as “Agreement Date” — which is useful contextual information even though the primary features used for classification are TF-IDF weighted terms rather than length.

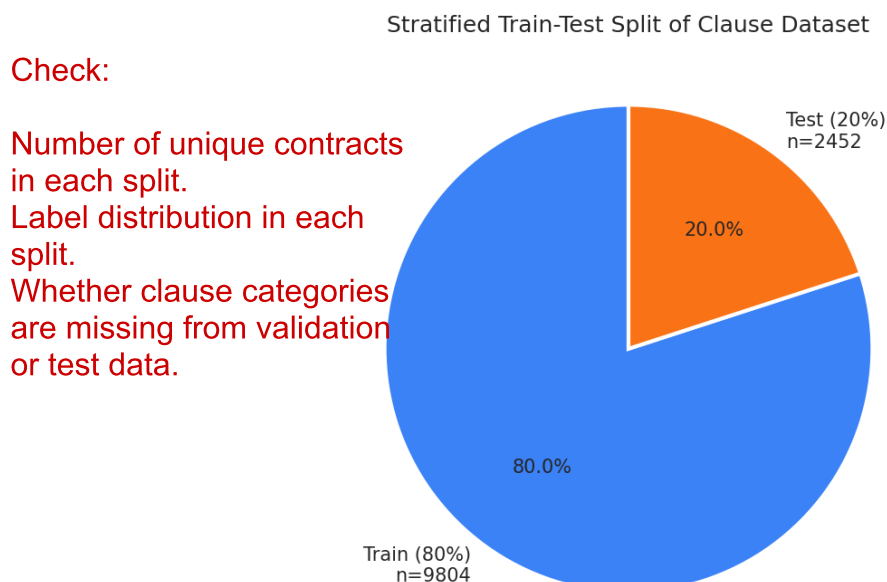


Figure 3.16: Planned stratified train/test split of the clause dataset

A stratified 80:20 train/test split (`random_state = 42`) is planned for the modelling stage so that every one of the 41 categories — including the rarest ones — is represented proportionally in both the training and test sets.

3.4. Analysis of Architecture and Algorithms

3.4.1. Detailed Study Report of the Proposed Algorithm(s)

ClauseIQ combines a TF-IDF feature representation with four classical supervised classifiers for clause categorisation, and a TF-IDF + cosine-similarity module for intelligent search over the same feature space.

TF-IDF Vectorisation

Term Frequency–Inverse Document Frequency converts each clause into a sparse numeric vector where each dimension corresponds to a term (unigram or bigram); a term’s weight is high if it occurs often in that clause but rarely across the corpus, which surfaces category-discriminating vocabulary and down-weights generic boilerplate. TF-IDF is well suited to short, boilerplate-heavy legal clauses because it does not require the large labelled corpora that transformer embeddings need, and it produces an interpretable feature space (individual terms can be inspected, unlike dense neural embeddings).

Multinomial Naive Bayes

A probabilistic classifier that assumes feature (term) independence given the class. It is extremely fast to train, performs well on sparse high-dimensional TF-IDF matrices, and serves as a strong, simple baseline for multi-class text classification — consistent with its documented use as a classical feature-based baseline among the Legal NLP studies surveyed by Quevedo et al. (2024).

Support Vector Machine (Linear Kernel)

SVMs find the maximum-margin hyperplane separating classes and are known to perform particularly well on high-dimensional, sparse text data such as TF-IDF vectors, where the number of features (up to 5,000) is much larger than what many other classifiers can handle efficiently. A linear kernel and one-vs-rest scheme are used to extend the binary SVM to the 41-class problem.

Logistic Regression

A linear probabilistic classifier trained with multinomial (softmax) loss, offering class probabilities (useful for ranking candidate categories, not just a single hard prediction) and good interpretability through per-class term coefficients, at low computational cost.

Random Forest

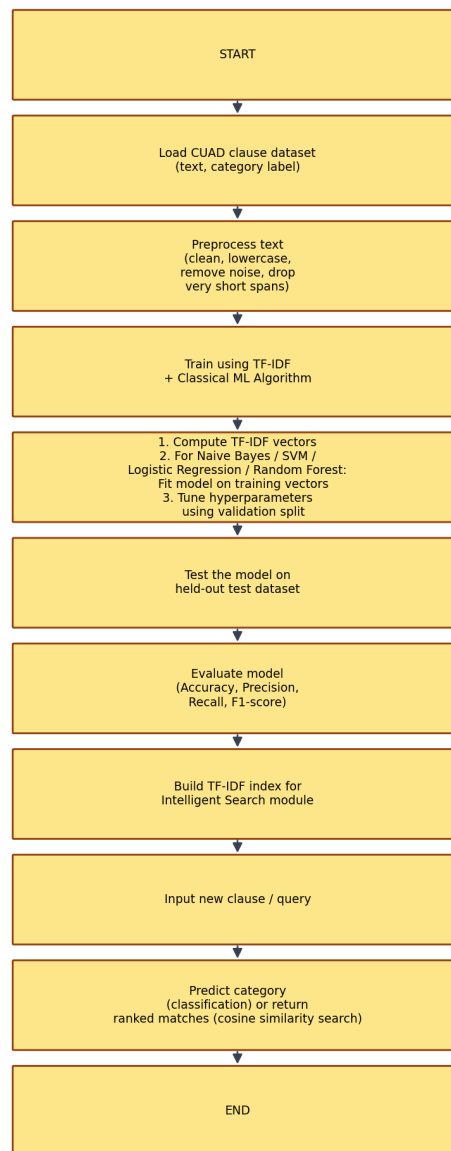
An ensemble of decision trees trained on bootstrap samples and random feature subsets, whose predictions are aggregated by majority vote. Random Forest can capture non-linear interactions between TF-IDF features and is naturally robust to noisy or irrelevant terms, at the cost of higher memory use on very high-dimensional sparse input.

Cosine Similarity (Intelligent Search)

For the search module, both the corpus of clauses and an incoming natural-language query are projected into the same TF-IDF space, and clauses are ranked by the cosine of the angle between their vector and the query vector. Cosine similarity is scale-invariant (it is unaffected by clause length) and is the standard similarity measure for sparse bag-of-words / TF-IDF retrieval, which is why it was selected over Euclidean distance for this task.

Justification (from Section 2 literature) Already mentioned in findings and may be avoided

- Quevedo et al. (2024), in their systematic mapping study of Legal NLP research (2015–2022), document multiclass legal-text classification as an established task family addressed historically by both classical, feature-based ML pipelines and deep-learning methods, and identify dataset scarcity and category-level imbalance as recurring challenges — directly supporting ClauseIQ’s use of TF-IDF-based classical classifiers with imbalance-aware training on CUAD’s 41 categories.
- Siino et al. (2025) confirm that TF-IDF-based classical pipelines are an established, interpretable starting point in legal NLP before moving to transformer-based approaches, and explicitly name CUAD as a standard contract-clause benchmark, matching ClauseIQ’s phased, scope-controlled strategy.
- Kutbi (2023) shows, on a legal-contract text-classification task (Limited Partnership Agreements) using the same broad TF-IDF-classification workflow, that lightweight NER-based preprocessing improves classical text-classifier accuracy while reducing feature dimensionality and preserving privacy — directly motivating ClauseIQ’s preprocessing design and its continued reliance on classical, feature-based classification rather than deep architectures.



Not readable

Figure 3.17: Activity diagram: TF-IDF + classical ML clause classification and search pipeline

Pseudocode: TF-IDF + Classifier Training

Input: Clause dataset D with text X and category labels Y (41 classes)

Output: Trained classifier + fitted TF-IDF vectorizer

```

function TRAIN_CLAUSE_CLASSIFIER(D, model_type):
    X_clean <- CLEAN_TEXT(D.text)
    tfidf <- FIT_TFIDF(X_clean, max_features=5000, ngram_range=(1,2))
    X_train, X_test, y_train, y_test <- STRATIFIED_SPLIT(tfidf, D.category,
    test_size=0.2)
    model <- INIT_MODEL(model_type, class_weight='balanced')    # NB / SVM / LR
    / RF
    model.fit(X_train, y_train)
    
```

```

y_pred <- model.predict(X_test)
metrics <- EVALUATE(y_test, y_pred) # macro-F1, per-class F1, accuracy
return model, tfidf, metrics

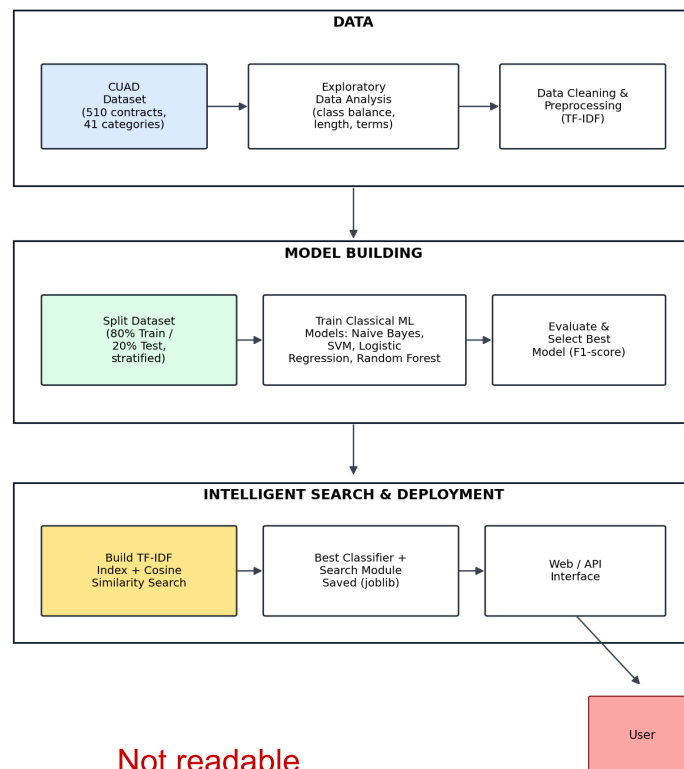
function SEARCH_CLAUSES(query, corpus_vectors, tfidf, top_k):
  q_vec <- tfidf.transform([query])
  scores <- COSINE_SIMILARITY(q_vec, corpus_vectors)
  return TOP_K_RANKED(scores, top_k)

```

This code comes in implementation section

3.5. Project Pipeline

3.5.1. Project Pipeline Diagram with Explanation



Not readable

Figure 3.18: ClauseIQ project pipeline

Data

The pipeline begins with the CUAD dataset (510 contracts, 41 clause categories), which is flattened from its SQuAD-style JSON format into a clause-level table as described in Section 3.1. Exploratory data analysis (class distribution, length distribution, missing-value and duplicate checks) is performed to understand the corpus, followed by data cleaning and TF-IDF-based preprocessing as described in Section 3.2.

Model Building

The cleaned, vectorised dataset is split into stratified training (80%) and testing (20%) subsets. Four classical ML models — Multinomial Naive Bayes, Linear SVM, Logistic Regression and Random Forest — are trained on the TF-IDF features with class-weighting to address the imbalance documented in Section 3.2.3, and compared using macro-F1, per-class F1 and accuracy to select the best-performing classifier.

Intelligent Search & Deployment

Independently of classification, a TF-IDF index of the full clause corpus is built to support cosine-similarity-based intelligent search: a free-text query is vectorised in the same TF-IDF space and the most similar clauses are ranked and returned. The selected classifier and the search index are planned to be persisted (via joblib) and exposed through a simple web/API interface so a user can classify a new clause or search the corpus for clauses matching a natural-language query.

3.6. Feasibility Analysis

Feasibility has been assessed across technical, economic and operational dimensions, consistent with the scope-controlled, classical-ML approach agreed with the project guide.

Technical Feasibility

- *Data processing:* CUAD is a public, well-documented, CC-BY-4.0-licensed dataset; it was downloaded directly from the official GitHub repository and successfully flattened into a clean, tabular clause dataset (13,823 rows) using standard Python libraries (pandas, json, re), confirming that data acquisition and preprocessing are fully achievable within the project's technical resources.
- *Machine learning:* TF-IDF vectorisation and the four chosen classifiers (Naive Bayes, SVM, Logistic Regression, Random Forest) are all implemented in scikit-learn, a mature, well-documented, free library; no custom low-level ML code or GPU hardware is required.
- *Search module:* cosine similarity over TF-IDF vectors uses the same scikit-learn feature space already built for classification, so no additional infrastructure (e.g. a dedicated vector database) is required at this project's scale (13,823 clauses).
- *Deployment:* the trained classifier and TF-IDF index can be serialized with joblib and served through a lightweight Python web framework (e.g. Flask), consistent with tooling already used elsewhere in the department's mini projects.

Economic Feasibility

- All software used — Python, pandas, scikit-learn, matplotlib/seaborn, Flask, joblib — is free and open-source; the CUAD dataset itself is free and openly licensed (CC BY 4.0).
- No paid annotation, cloud GPU, or third-party API costs are required for the classical ML approach adopted here, keeping the project's running cost close to zero beyond standard

computer hardware and development time.

- The resulting tool has clear practical value — faster clause triage for legal/compliance teams, students, and paralegals reviewing standard commercial contracts — which justifies the (minimal) development effort.

Operational Feasibility

- End users (e.g. a paralegal or contract reviewer) only need to paste clause text or a search query into a simple web interface; no legal-technology or machine-learning expertise is required to operate the system.
- Because the underlying models are lightweight classical classifiers rather than large neural networks, inference is fast enough for interactive, single-clause or single-query use on ordinary hardware, without needing a GPU or specialised serving infrastructure.
- The system’s scope is intentionally bounded to the 41 CUAD categories and to classical ML, which keeps the operational surface small and maintainable within a single-semester mini project timeline, while leaving a clear, well-justified extension path (transformer / RAG-based deep learning) for a future main project.